

DS03A - Structures IF / FOR / WHILE

1.

```
def fonc1(a,b):
    if a < b:
        return b - a
    else:
        return a * b
```

Que renvoie la commande `fonc1(4,3)` ?

Que renvoie la commande `fonc1(3,4)` ?

2.

```
def fonc2(n):
    a = 1
    for i in range(n):
        a = a * 2
    return a
```

Que renvoie la commande `fonc2(4)` ?

3.

```
def fonc3(une_liste):
    a = 0
    for elt in une_liste:
        a = a + elt
    return a
```

Quelle est la valeur renvoyée par `fonc3([1,3,4,6])` ?

4.

```
def fonc4(une_liste):
    n = len(une_liste)
    sortie = []
    for i in range(n):
        if une_liste[i] % 2 == 0:
            sortie.append(i)
    return sortie
```

Quelle est la valeur renvoyée par `fonc4([1,3,4,6])` ?

5.

```
def fonc5(n):
    a = 10
    while n > 0:
        a = a + 1
        n = n - 1
    return a
```

Quelle est la valeur renvoyée par `fonc5(6)` ?

6. En utilisant vos connaissances et les structures que vous connaissez, proposez une fonction nommée `recherche_maximum(une_liste:list)` qui renvoie la valeur maximum d'une liste passée en argument.

```
def recherche_maximum(une_liste):
    ''' Fonction renvoyant la valeur maximale d'une liste
    Entrée:
    - une_liste: liste de n valeurs de type int
    Sortie:
    - maximum: maximum de la liste (type int) '''

    maximum = une_liste[0]

    for i in range(1, len(une_liste)):
        if une_liste[i] > maximum:
            maximum = une_liste[i]

    return maximum

ma_liste = [3,6,8,1,9,4,7]
assert recherche_maximum(ma_liste) == 9
```

7. En utilisant vos connaissances et les structures que vous connaissez, proposez une fonction nommée `somme_liste_pairs(une_liste:list)` qui renvoie la somme des éléments pairs d'une liste passée en argument.

```
def somme_liste_pairs(une_liste):  
  
    return somme_pairs  
  
ma_liste = [3,6,8,1,9,4,7]  
assert somme _liste_pairs(ma_liste) == 18
```

DS03B - Structures IF / FOR / WHILE

1.

```
def fonc1(a,b):
    if a < b:
        return b - a
    else:
        return a * b
```

Que renvoie la commande `fonc1(2,3)` ?

Que renvoie la commande `fonc1(3,2)` ?

2.

```
def fonc2(n):
    a = 1
    for i in range(n):
        a = a * 2
    return a
```

Que renvoie la commande `fonc2(3)` ?

3.

```
def fonc3(une_liste):
    a = 0
    for elt in une_liste:
        a = a + elt
    return a
```

Quelle est la valeur renvoyée par `fonc3([1,2,4,6])` ?

4.

```
def fonc4(une_liste):
    n = len(une_liste)
    sortie = []
    for i in range(n):
        if une_liste[i] % 2 == 0:
            sortie.append(i)
    return sortie
```

Quelle est la valeur renvoyée par `fonc4([1,2,4,6])` ?

5.

```
def fonc5(n):
    a = 10
    while n > 0:
        a = a + 1
        n = n - 1
    return a
```

Quelle est la valeur renvoyée par `fonc5(5)` ?

6. En utilisant vos connaissances et les structures que vous connaissez, proposez une fonction nommée `recherche_minimum(une_liste:list)` qui renvoie la valeur minimum d'une liste passée en argument.

```
def recherche_minimum(une_liste):
    ''' Fonction renvoyant la valeur maximale d'une liste
    Entrée:
    - une_liste: liste de n valeurs de type int
    Sortie:
    - minimum: minimum de la liste (type int) '''

    minimum = une_liste[0]

    for i in range(1, len(une_liste)):
        if une_liste[i] < minimum:
            minimum = une_liste[i]

    return minimum

ma_liste = [3,6,8,1,9,4,7]
assert recherche_minimum(ma_liste) == 1
```

7. En utilisant vos connaissances et les structures que vous connaissez, proposez une fonction nommée `somme_liste_impairs(une_liste:list)` qui renvoie la somme des éléments impairs d'une liste passée en argument.

```
def somme_liste_impairs(une_liste):  
  
    return somme_impairs  
  
ma_liste = [3,6,8,1,9,4,7]  
assert somme _liste_impairs(ma_liste) == 20
```